HUAWEI

Security Level: SECRET

Dynamic PUCT: Global Prior-Guided Test-Time Search

NetMind Canada

www.huawei.com

Mohammad Hossein Moslemi (m84449988) /

Standard PUCT

$$\mathrm{PUCT}(s) = \underbrace{\frac{W(s)}{m_s}}_{\substack{\text{exploitation:}\\ \text{mean value of subtree}}} + c \cdot P(s \mid p) \cdot \underbrace{\frac{\sqrt{m_p}}{1 + m_s}}_{\substack{\text{exploration:}\\ \text{favors rarely visited children}}}$$

- $W(s) = \sum_{y \in \mathcal{T}(s)} Q(y)$: accumulated value of the subtree rooted at s
- $m_s = |\mathcal{T}(s)|$: subtree size (used in place of visit count)
- $P(s \mid p) = \mathrm{softmax}\big(\{\, Q(s') \mid s' \in \mathcal{S}(p) \,\}\big)_s$: child prior

D-PUCT$\rightarrow$ Stands for Dynamic-PUCT. Replace exploitation term by $\max_{s' \in \mathcal{T}} Q(s')$, and the $P(s|p)$ by the "Global Node prior" and "Virtual Child".

Global Node Logit $L_D(s)$

All nodes in the search tree are stored in a flat dataset D. For every node $s \in D$, we compute two dataset-level signals:

$$\text{Rank signal} \longrightarrow \widetilde{R}(s) = \frac{R(s) - \mu_{D,R}}{\sigma_{D,R}}, \qquad \text{ranking is based on } W_m(s) \tag{1}$$

$$\text{Elo debate signal} \longrightarrow \widetilde{E}(s) = \frac{E(s) - \mu_{D,E}}{\sigma_{D,E}}. \tag{2}$$

The two signals define a **global node logit**:

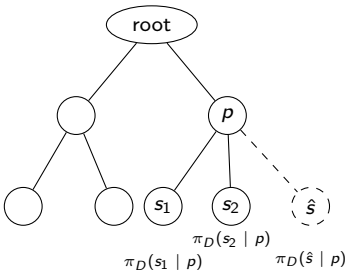
$$\boxed{L_D(s) = \alpha\, \widetilde{R}(s) + (1 - \alpha)\, \widetilde{E}(s)}$$

Global scoring, local normalization

$L_D(s)$ is computed relative to all nodes in D, but it is **not normalized over** D. When a parent p is visited, only its currently available actions are normalized through a parent-local softmax.

Virtual Child & Parent-Local Prior

Let $\mathcal{C}(p)$ be real child of p, and let $\hat{s}$ denote the virtual action "generate a new child."



Virtual-child logit

$$\mu_L(p) = \frac{1}{|\mathcal{C}(p)|} \sum_{u \in \mathcal{C}(p)} L_D(u),$$

$$\sigma_L(p) = \sqrt{\frac{1}{|\mathcal{C}(p)|} \sum_{u \in \mathcal{C}(p)} \big(L_D(u) - \mu_L(p)\big)^2},$$

$$L_p(\hat{s}) = \mu_L(p) + \lambda\, \sigma_L(p).$$

Virtual Child & Parent-Local Prior (cont.)

Define the available action set $\mathcal{A}(p) = \mathcal{C}(p) \cup \{\hat{s}\}$, and assign every action a logit in the same space:

$$L_p(a) = \begin{cases} L_D(a), & a \in \mathcal{C}(p), \\ \mu_L(p) + \lambda \sigma_L(p), & a = \hat{s}. \end{cases}$$

$$\boxed{\pi_D(a \mid p) = \frac{\exp\big(L_p(a)/\tau\big)}{\sum_{b \in \mathcal{A}(p)} \exp\big(L_p(b)/\tau\big)}}$$

- Small τ: concentrates probability on strong actions.
- Large τ: produces a flatter local prior.

D-PUCT Selection Procedure

At parent p, each available action $a \in \mathcal{A}(p)$ is assigned

$$\text{D-PUCT}(p, a) = V(p, a) + c\, \pi_D(a \mid p) \frac{\sqrt{m_p}}{1 + m_{p,a}}.$$

Selection procedure

1. Start at the root node.
2. At the current node p, select

$$a^\star = \arg \max_{a \in \mathcal{A}(p)} \text{D-PUCT}(p, a).$$

3. Apply the selected action:
   - If $a^\star$ is a non-leaf real child, move to that child and repeat the selection step.
   - If $a^\star$ is a leaf child, generate k children.
   - If $a^\star = \hat{s}$, generate one new child of p.

HUAWEI

Security Level: SECRET

Adaptive Memory for Test-Time

NetMind Canada

www.huawei.com

Mohammad Hossein Moslemi (m84449988) / July 22, 2026

Test-Time Learning with Adaptive Memory (EACL'26)

- Maintains an evolving external memory containing useful strategies, insights, and solutions
- No parameter updates, ground-truth labels, or human feedback required
- **BL:** solves every problem independently
- **FH:** provides the entire previous history
- **DC-RS:** Their Dynamic-Cheetsheet method. They reused the same LLM to Curate the memory using a different prompt.

BL (Baseline)

for $i \in [1, \ldots, n]$ **do**
  $\tilde{y}_i = \mathsf{Gen}(x_i)$ ▷ *Solution generation*
end for

FH (Full History)

for $i \in [1, \ldots, n]$ **do**
  $M_i = \mathsf{Concat}(\{(x_j, \tilde{y}_j)\}_{j<i})$ ▷ *Append full history*
  $\tilde{y}_i = \mathsf{Gen}(x_i, M_i)$ ▷ *Solution generation*
end for

DC-RS (Retrieval and Synthesis)

$M_0 \leftarrow \emptyset$ ▷ *Memory initialization*
for $i \in [1, \ldots, n]$ **do**
  $R_i = \mathsf{Retr}(x_i, \{(x_j, \tilde{y}_j)\}_{j<i}, k)$ ▷ *Retrieval*
  $M_i = \mathsf{Cur}(M_{i-1}, x_i, R_i)$ ▷ *Memory curation*
  $\tilde{y}_i = \mathsf{Gen}(x_i, M_i)$ ▷ *Solution generation*
end for

Retrieve top k previous with output → Curate → Generate

Results on AIME

- AIME is a high-school mathematics competition covering algebra, combinatorics, number theory, geometry, and probability

Tasks	Claude 3.5 Sonnet			GPT-4o		
	BL	FH	DC-Cu.	BL	FH	DC-RS
AIME 2024	23.3	26.7	**50.0**	20.0	13.3	**40.0**
AIME 2025	6.7	6.7	**36.7**	6.7	3.3	**20.0**

GENERAL META-REASONING STRATEGIES
<memory_item>

<description>
Systematic Problem Analysis Framework (*Reference: Q1-Q20*)
For complex mathematical problems:
1. State problem requirements clearly
2. List key observations and theorems applicable
3. Identify patterns and relationships
4. Break into manageable sub-problems
5. Verify against examples
6. Consider computational approach when analytical solution is complex
7. For grid problems, analyze movement patterns and symmetries
8. For combinatorial problems, use appropriate counting techniques
9. Implement verification code when possible
10. Consider edge cases and constraints
11. For grid coloring problems, consider row/column patterns
</description>

<example>
Example application:
1. Requirements: list all given conditions
2. Observations: identify applicable theorems
3. Patterns: look for structural relationships
4. Sub-problems: break into steps
5. Verification: test against examples
6. Implementation: use Python for verification
</example>

</memory_item>
Count: 20

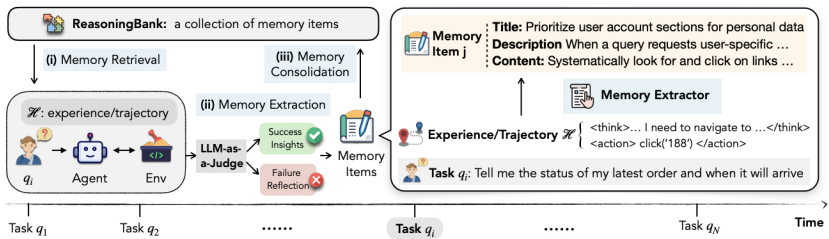
ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory (ICLR'26, Google)

Tasks $q_1, \ldots, q_N$ arrive in a stream; no ground-truth labels. The agent must evolve using only its own past trajectories and self-verification.

Gap in prior literature: raw trajectories (Synapse) or success-only workflows (AWM)

- No distillation and mostly raw trajectories only
- Failures, a rich source of preventative lessons, are discarded

ReasoningBank: distills structured memory items from *both* self-judged successful and failed trajectories, forming a closed loop: (i) **retrieval** → (ii) **extraction** → (iii) **consolidation** (add)



How the Memory Is Constructed

Memory item schema (Markdown, human- and machine-readable):

- **Title** (strategy identifier)
- **Description** (one sentence)
- **Content** (1–3 sentences of distilled reasoning / pitfalls)

Extraction pipeline (per completed task):

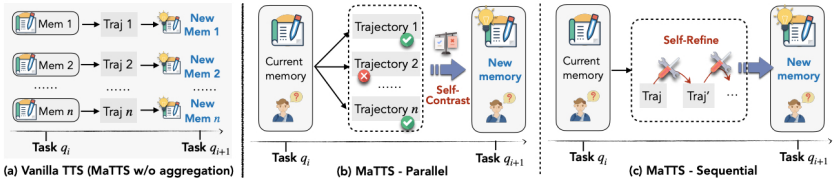
1. **LLM-as-Judge** (same backbone) labels trajectories Success/Failure with no ground truth.
2. **Dual prompts**:
   - success $\rightarrow$ explain *why it worked*, extract transferable strategies
   - failure $\rightarrow$ reflect on the cause, extract preventative lessons.

Retrieval: embed the new query, cosine similarity over the pool, take items of the top-k experiences, inject into the system prompt before acting.

Consolidation: Minimal, new items are simply appended to a pool (no pruning or merging).

MaTTS: Memory-Aware Test-Time Scaling

Instead of extracting memory from each rollout independently (a) do it in **MaTTS Parallel** with k rollouts per task to keep consistent patterns and filter spurious ones or **MaTTS Sequential** with *self-refinement* within one trajectory that intermediate correction notes become memory signals.



on WebArena benchmark. Success rate (SR ↑) and the number of steps (Step ↓) are reported on 5 subsets for 3 different backbone LLMs.

Models	Shopping (187)		Admin (182)		Gitlab (180)		Reddit (106)		Multi (29)		Overall (684)	
	SR	*Step*	*SR*	*Step*	*SR*	*Step*	*SR*	*Step*	*SR*	*Step*	*SR*	*Step*
Gemini-2.5-flash												
No Memory	39.0	8.2	44.5	9.5	33.9	13.3	55.7	6.7	10.3	10.0	40.5	9.7
Synapse	40.6	7.0	45.1	9.1	35.6	13.0	59.4	6.5	10.3	10.5	42.1	9.2
AWM	44.4	7.0	46.7	8.8	37.2	13.2	62.3	6.1	3.4	7.7	44.1	9.0
ReasoningBank	**49.7**	**6.1**	**51.1**	**8.2**	**40.6**	**12.3**	**67.0**	**5.6**	**13.8**	8.8	**48.8**	**8.3**
+MaTTS	53.0	6.3	53.8	7.6	42.8	11.9	70.8	5.4	17.2	8.0	51.8	7.9

Memory Extractor prompt

System Instruction

You are an expert in web navigation. You will be given a user query, the corresponding trajectory that represents **how an agent successfully accomplished the task**.

Guidelines
You need to extract and summarize useful insights in the format of memory items **based on the agent's successful trajectory**.
The goal of summarized memory items is to be helpful and generalizable for future similar tasks.

Important notes
- You must first **think why the trajectory is successful**, and then summarize the insights.
- You can extract *at most 3* memory items from the trajectory.
- You must not repeat similar or overlapping items.
- Do not mention specific websites, queries, or string contents, but rather focus on the generalizable insights.

Output Format
Your output must strictly follow the Markdown format shown below:
```
# Memory Item i
## Title <the title of the memory item>
## Description <one sentence summary of the memory item>
## Content <1-3 sentences describing the insights learned to successfully accomplishing the task> ```
```

Input Prompt

Query: <user query>

Trajectory: <trajectory that completes the query>

System Instruction

You are an expert in web navigation. You will be given a user query, the corresponding trajectory that represents **how an agent attempted to resolve the task but failed**.

Guidelines
You need to extract and summarize useful insights in the format of memory items **based on the agent's failed trajectory**.
The goal of summarized memory items is to be helpful and generalizable for future similar tasks.

Important notes
- You must **first reflect and think why the trajectory failed**, and then summarize what lessons you have learned or strategies to prevent the failure in the future.
- You can extract *at most 3* memory items from the trajectory.
- You must not repeat similar or overlapping items.
- Do not mention specific websites, queries, or string contents, but rather focus on the generalizable insights.

Output Format
Your output must strictly follow the Markdown format shown below:
```
# Memory Item i
## Title <the title of the memory item>
## Description <one sentence summary of the memory item>
## Content <1-3 sentences describing the insights learned to successfully accomplishing the task> ```
```

Input Prompt

Query: <user query>

Trajectory: <trajectory that completes the query>

Test-Time Self-Distillation (SDPO), ICLR'26 workshop

Problem: With a reward 0 on every attempt → zero advantage, no learning signal until the first solution is found for RLVR (e.g., GRPO).

Key idea: the model is its own teacher

- *Student* $\pi_\theta(y \mid x)$: the model answering cold
- *Self-teacher* $\pi_\theta(y \mid x, f)$: the *same weights* re-scoring the failed attempt with environment feedback f (traceback, failing test) in context
- With f visible, per-token probabilities shift exactly at the buggy tokens; elsewhere they barely move ⇒ dense credit assignment for free

SDPO loop

1. Sample G attempts, collect feedback
2. Re-score each attempt under the feedback-augmented prompt
3. Minimize $\sum_t \mathrm{KL}\big(\pi_\theta(\cdot \mid x, y_{<t}) \,\|\, \mathrm{sg}(\pi_\theta(\cdot \mid x, f, y_{<t}))\big)$ ‘

Connection to Our Work

SDPO is not a memory mechanism but a way to ensure that, regardless of the RL algorithm we choose, learning occurs from both positive scores and failed attempts.

Caveat: We should not run RL in a stepwise manner. Standard RL updates change the underlying π, making SDPO inapplicable to the new π. Instead, SDPO should be integrated directly into whichever RL method we use.

HUAWEI

Security Level: SECRET

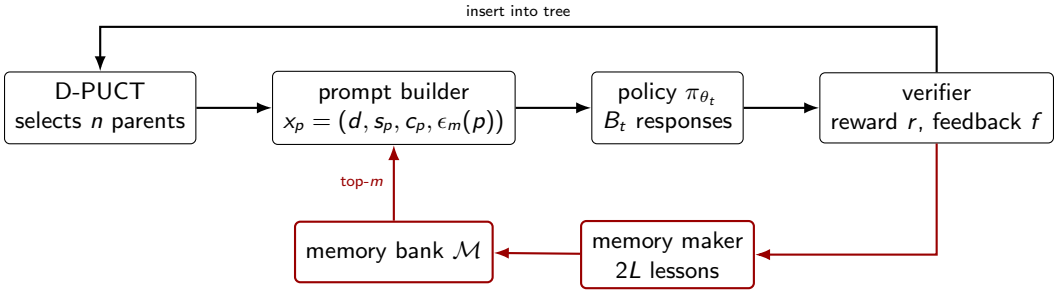
Memory-Augmented Search with Failure Feedback

NetMind Canada

www.huawei.com

Mohammad Hossein Moslemi (m84449988) / August 19, 2026

Where Memory Sits in the Loop



- Same LLM backbone as the generator, dedicated extraction prompts.
- Two calls per step: one over the successes, one over the failures.

Version 1

Partition step t's responses by verifier reward, subsample each group to k, and send each to the backbone in one call:

$$\mathcal{L}_t^{+} = \mathrm{LLM}_{\mathrm{mem}}\big(\mathrm{prompt}^{+}, \mathcal{S}_t^{(k)}, \mathcal{C}_t\big), \qquad \mathcal{L}_t^{-} = \mathrm{LLM}_{\mathrm{mem}}\big(\mathrm{prompt}^{-}, \mathcal{F}_t^{(k)}, \mathcal{C}_t\big)$$

The bank is additive but deduplicated at entry:

$$\mathcal{M}_t = \mathcal{M}_{t-1} \cup \mathcal{D}_\tau\big(\mathcal{L}_t^{+} \cup \mathcal{L}_t^{-}\big)$$

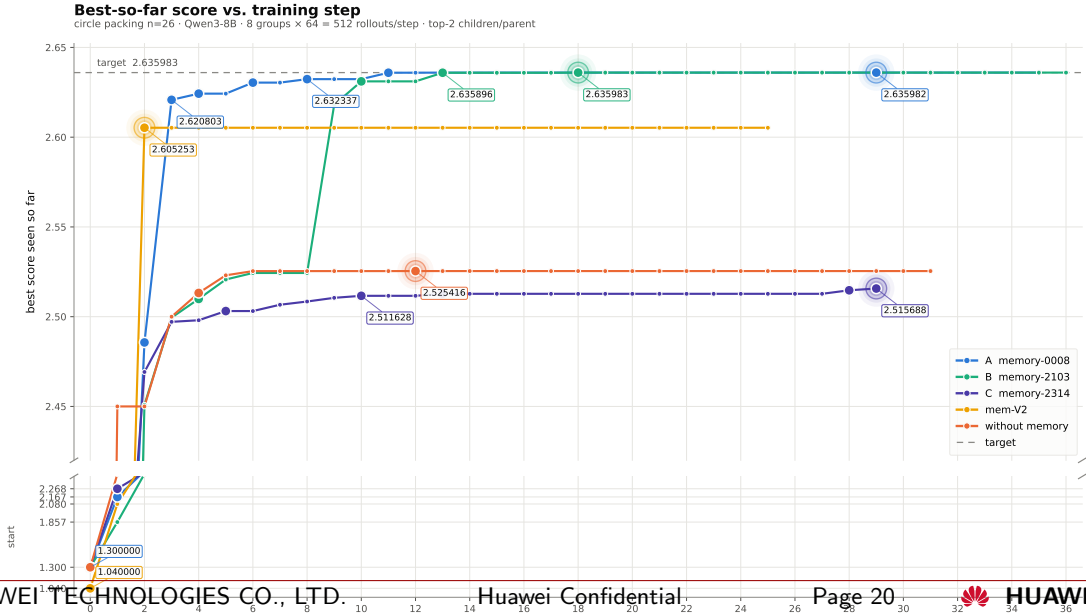
- Before expanding a parent p, retrieve m lessons by cosine similarity, so all children share one set of lessons.

Experimental Setup

Circle packing, $n = 26$. Best known value 2.635983. Qwen3-8B with LoRA, 30 steps of $B_t = 512$ rollouts, 8 parents $\times$ 64 children, PUCT.

Run	Memory	Steps	Rollouts	Best score	Valid	Valid$^+$	Lessons
MEM-A	yes	36	18,432	2.635982	0.551	0.523	194
MEM-B	yes	37	18,944	**2.635983**	0.734	0.709	185
MEM-C	yes	30	15,360	2.515688	0.745	0.645	162
NO-MEM	no	32	16,384	2.525416	0.646	0.594	—

- MEM-A and MEM-C are some configuration differences in A,B, and C.


Best-so-far score vs. training step
circle packing n=26 · Qwen3-8B · 8 groups × 64 = 512 rollouts/step · top-2 children/parent
best score seen so far
target 2.635983
2.620803
2.605253
2.632337
2.635896
2.635983
2.635982
2.525416
2.511628
2.515688
A memory-0008
B memory-2103
C memory-2314
mem-V2
without memory
target
start
1.300000
1.040000

Findings

- 87%–89% of lessons **never retrieved**; the top five took 60%–77% of all retrievals.
- Code is 23%–34% of each bank but takes 88%–96% of retrieval mass.
- All 48 lessons on global search, ensembles, and restarts: **zero retrievals**. MEM-B escaped conditioned only on lessons describing the plateau it was leaving.

Reason

The query barely changes across parents, so the cosine term barely varies

Examples of lessons

ID	Run	Injected	Lesson text

979e6dc0	C	14,336 (19.3%)	`rows = int(np.ceil(np.sqrt(n)));`
			`x = 0.5 + (col - (cols-1)/2) * (1/(cols-1));`
			`y = 0.5 + (row - (rows-1)/2) * (1/(rows-1));`
			`if row % 2 == 1:  y += np.sqrt(3)/2 / 2`

1644ca11	B	17,600 (19.1%)	`dist = np.linalg.norm(centers[i] - centers[j]);` `radii[i] = min(radii[i], dist / 2)`
d3b941ec	A	13,888 (15.5%)	Clamp circle positions to the unit square bounds: `x = max(min(x,1),0)`.
3a29df87	A	**0 (0.0%)**	Initialize centers with a hexagonal grid layout for *n* circles, adjusting spacing to fit the unit square.

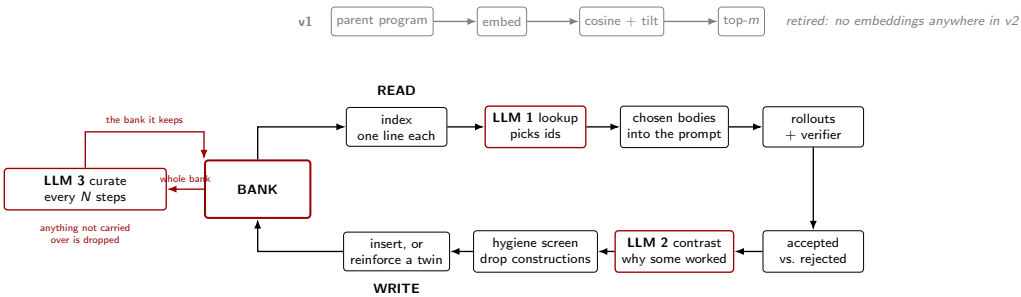
Memory Version 2 — What a Lesson May Contain

A lesson may describe an **operation**.
It may never hand over a **construction**.

`scope`	Content	Code
`local`	repair, check, diagnostic, guard	up to 4 lines
`global`	starting configuration, formulation, method family	**none**

The contrastive call asks why some attempts succeeded *and others failed*, with each child shown against its parent's score.

Version 2 — One Bank, Read and Written

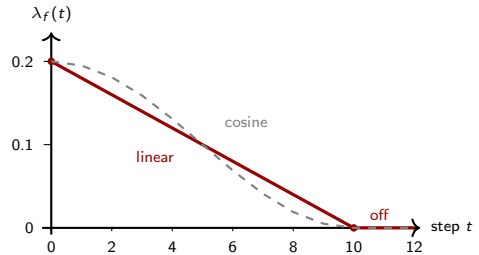


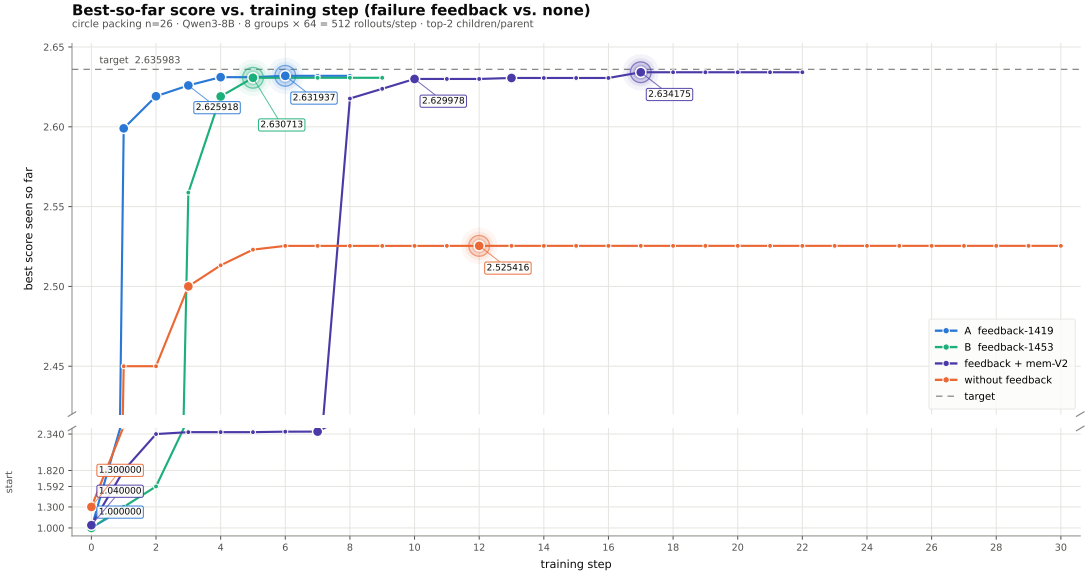
Learning from Failures

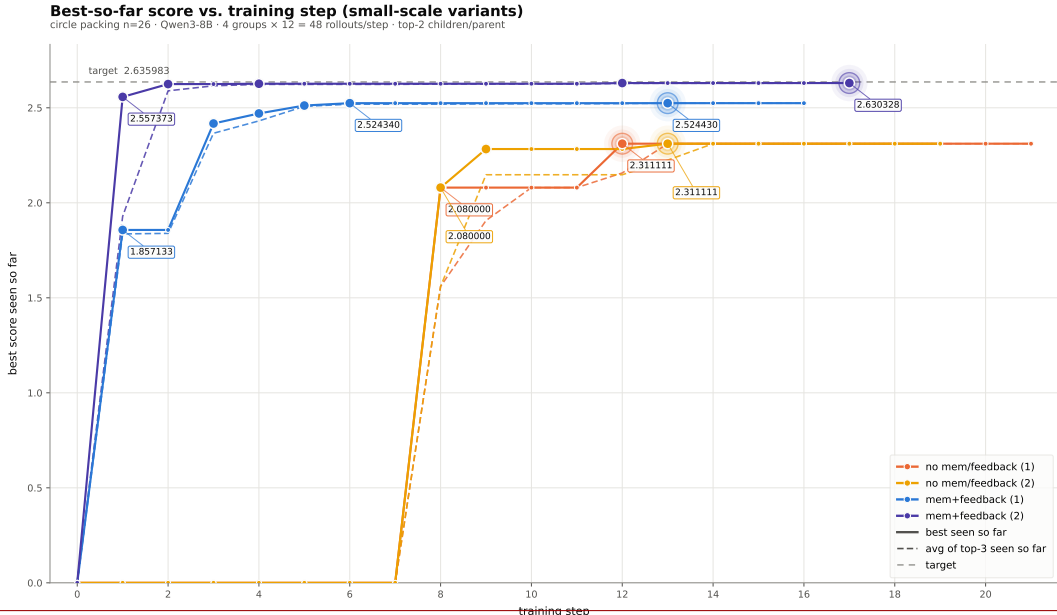
A scalar reward says a rollout failed, not *which tokens* caused it. Failures are 50%–70% of init step's, and constant-reward groups were skipped entirely. Reuse them as feedback.

$$A^{\mathrm{fb}}_{i,\ell} = \log q_{\tilde{\theta}}\big(y_{i,\ell} \mid x_p, f_i, y_{i,<\ell}\big) - \log \pi_{\tilde{\theta}}\big(y_{i,\ell} \mid x_p, y_{i,<\ell}\big), \qquad A_{i,\ell} = A^{\mathrm{rew}}_i + \lambda_f(t)\, d_i\, A^{\mathrm{fb}}_{i,\ell}$$

- $q_{\tilde{\theta}}$ is $\pi_{\tilde{\theta}}$ on $\mathrm{reprompt}(x_p, f_i)$.
- Positive where the feedback makes the token *more* plausible, negative where it makes it less so.
- Constant-reward groups become trainable: $A^{\mathrm{rew}}_i = 0$ but A^{fb} is not.




Best-so-far score vs. training step (failure feedback vs. none)
circle packing n=26 · Qwen3-8B · 8 groups × 64 = 512 rollouts/step · top-2 children/parent
target 2.635983
2.625918
2.630713
2.631937
2.629978
2.634175
2.525416
best score seen so far
training step
start
1.300000
1.040000
1.000000
A feedback-1419
B feedback-1453
feedback + mem-V2
without feedback
target


Best-so-far score vs. training step (small-scale variants)
circle packing n=26 · Qwen3-8B · 4 groups × 12 = 48 rollouts/step · top-2 children/parent
target 2.635983
2.557373
1.857133
2.524340
2.524430
2.630328
2.311111
2.311111
2.080000
2.080000
best score seen so far
training step
no mem/feedback (1)
no mem/feedback (2)
mem+feedback (1)
mem+feedback (2)
best seen so far
avg of top-3 seen so far
target

HUAWEI

Security Level: SECRET

Max-Seeking Memory and Adaptive Failure Feedback

NetMind Canada

www.huawei.com

Mohammad Hossein Moslemi (m84449988) / August 26, 2026

Discovery Objective and Notation

Symbols used in the next slides

x	selected parent program/state
$\pi_\theta(\cdot \mid x)$	current program generator
Y	one generated candidate program
$R(Y)$	verified reward (larger is better)
K	rollout budget for this parent

The quantity we want to improve

$$Y_1, \ldots, Y_K \sim \pi_\theta(\cdot \mid x),$$

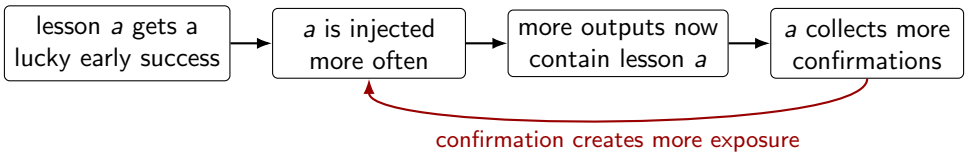
$$J_K(\pi_\theta \mid x) = \mathbb{E}\left[\max_{1 \le i \le K} R(Y_i)\right].$$

J_K is the expected reward of the **best** candidate among the K attempts.

Max-seeking, not mean-seeking

Improving the typical program is useful only if it does not remove the rare programs that can produce a new best result.

Memory Collapse



Suppose lesson "*a*" is injected 20 times and lesson "*b*" only 4 times. Even if the lessons are equally useful, "*a*" has many more opportunities to be reused, confirmed, or followed by a valid output. A count-based score will therefore favor "*a*" partly because the selector already favored "*a*".

What is missing

For the *same parent* and the *same number of attempts*, would lesson "*a*" produce a better maximum than lesson "*b*" or no memory?

Memory Collapse

$p_t(m)$ is the probability of injecting lesson m at update t; $u_t(m)$ is its observed proxy score; and $\eta > 0$ controls how strongly the selector reacts to score differences.

Selection update

$$p_{t+1}(m) = \frac{p_t(m)e^{\eta u_t(m)}}{\sum_j p_t(j)e^{\eta u_t(j)}}.$$

Compare lessons a and b

$$\frac{p_{t+1}(a)}{p_{t+1}(b)} = \frac{p_t(a)}{p_t(b)} e^{\eta[u_t(a)-u_t(b)]}.$$

The exponential multiplies the old selection odds by a factor determined by the score gap.

For $\eta = 1$ and $u_t(a) - u_t(b) = 0.4$, one update multiplies the odds for a by $e^{0.4} \approx 1.49$. If that biased gap lasts for $T = 10$ updates, the total factor is $e^4 \approx 54.6$ without proving that a improves final reward.

Feedback Collapse

For rollout i and token ℓ, $A_{i,\ell} = A_i^{\text{rew}} + \lambda_f d_i A_{i,\ell}^{\text{fb}}$.

A_i^{rew} is scientific reward advantage; $A_{i,\ell}^{\text{fb}}$ is the token-level repair signal; $d_i = 1$ only for a failed rollout; and λ_f is feedback strength.

Example: every candidate is invalid

All candidates receive the same reward, so $A_i^{\text{rew}} \approx 0$. The verifier nevertheless gives detailed syntax, timeout, or shape corrections, so $A_{i,\ell}^{\text{fb}}$ remains informative and dense.

What the optimizer then learns

The update is driven almost entirely by repairs. It favors familiar programs that are easy to make valid, even if unusual programs are the only route to a large scientific reward.

Repair answers "is it valid?", not "how good can it become?"

Memory Bandit

$$M_n(m \mid x) = \mathbb{E}\left[\max_{1 \le j \le n} R(Y_j) \;\middle|\; x,\ m \text{ is injected}\right].$$

generate n candidates from parent x, keep the largest reward, and average that maximum over repeated batches. The symbol $m = \emptyset$ means no memory.

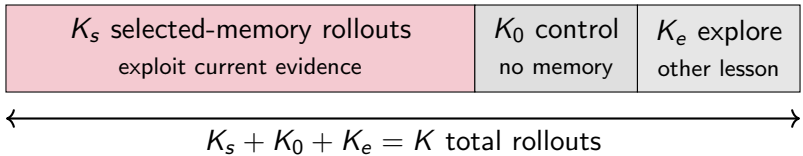
Subtract the no-memory counterfactual

$$\Delta_m^{(n)}(x) = M_n(m \mid x) - M_n(\emptyset \mid x).$$

Both terms use the same parent and the same n, so the difference isolates the effect of injecting lesson m.

$\Delta_m^{(n)} > 0$	$\Rightarrow$	memory improves the attainable maximum
$\Delta_m^{(n)} \approx 0$	$\Rightarrow$	memory adds no measured value
$\Delta_m^{(n)} < 0$	$\Rightarrow$	memory suppresses better directions

Memory Bandit



Selected arm s

Most rollouts may use the currently preferred lesson so useful memory still accelerates search.

Control arm 0

A small permanent no-memory arm estimates what would have happened without memory.

Exploration arm e

A small permanent exploratory arm tests lessons that are promising or uncertain.

Normalize Values

Raw maxima are unfair

Even if two arms have the same reward distribution, the maximum of 10 samples is usually larger than the maximum of 3 samples. This is the *best-of-more* advantage.

Suppose an arm produced k rewards $\mathbf{r} = (r_1, \ldots, r_k)$. To compare it at size $n \le k$, use

$$\widehat{M}_n(\mathbf{r}) = \mathbb{E}_S\left[\max_{j \in S} r_j\right], \qquad S \sim \text{Uniform}\{S \subseteq \{1, \ldots, k\} : |S| = n\}.$$

Example

For rewards $(1, 2, 3)$ and $n = 2$, the possible subsets have maxima $2, 3, 3$. Therefore

$$\widehat{M}_2(1, 2, 3) = \frac{2 + 3 + 3}{3} = \frac{8}{3}.$$

Matched comparison

For memory and control arms, set $n = \min(K_m, K_0)$ and compute

$$\widehat{\Delta}_m = \widehat{M}_n(\mathbf{r}_m) - \widehat{M}_n(\mathbf{r}_0).$$

Upper confidence bound for memory

$$\mathrm{UCB}(m) = \overline{\Delta}_m + c\, s_R \sqrt{\frac{\log(1+N)}{1+n_m}}.$$

Measured-value term

$\overline{\Delta}_m$ is lesson m's average matched uplift. A reliably positive lesson remains attractive even after many tests.

Uncertainty term

n_m is the number of matched tests of m; $N = \sum_m n_m$ is all matched tests. The bonus is large when n_m is small and shrinks as evidence arrives.

Adaptive Feedback

Validity controller

Let v_t be the fraction of valid programs at step t. Define

$$\lambda_{\text{eff}}(t) = \lambda_{\text{sched}}(t) g(v_t),$$

$$g(v) = \begin{cases} 1, & v \le v_{\text{floor}}, \\ \dfrac{v_{\text{target}} - v}{v_{\text{target}} - v_{\text{floor}}}, & v_{\text{floor}} < v < v_{\text{target}}, \\ 0, & v \ge v_{\text{target}}. \end{cases}$$

Read it as three regimes

- Below v_{floor}: full scheduled feedback.
- Between the thresholds: linearly decreasing feedback.
- Above v_{target}: feedback is off.

Weights

$\lambda_{\text{sched}}(t)$ is the planned weight before gating; $\lambda_{\text{eff}}(t)$ is the weight actually used.

Feedback is strong only while invalidity is the bottleneck.

Capped feedback restores tail quality and validity

Seed 42 — 12 steps — 64 rollouts/step — 768 rollouts/run

Arm	Best $C_5 \downarrow$	Valid overall	Valid at step 11	Distinct constructions
Plain	0.381327	48.0%	84.4%	175/369 (47%)
Memory only	**0.381264**†	41.5%	51.6%	**274/319 (86%)**
Feedback only (capped)	**0.381289**	59.5%	90.6%	222/457 (49%)
Memory + feedback	0.381531	**62.5%**	**93.8%**	205/480 (43%)

Capped feedback works

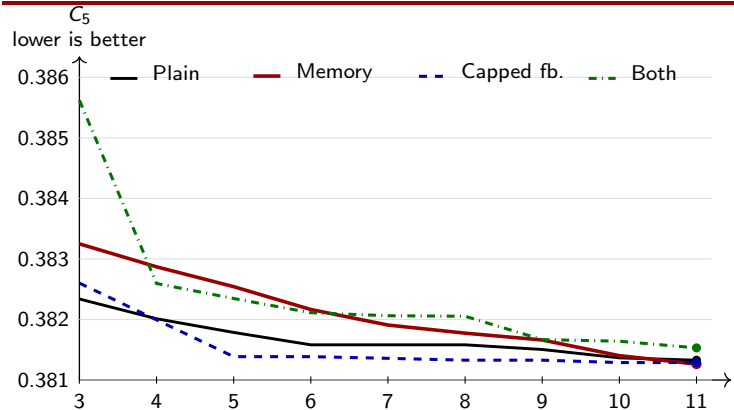
It beats plain on the best score while raising overall validity by 11.5 percentage points.

Both wins reliability

93.8% final-step validity, but a weaker extreme solution than plain or memory-only.

Feedback-only uses $B_{\mathrm{fb}} = 16$ and $C_{\mathrm{signature}} = 4$; the other rows retain their original settings. †Best rollout; the archive kept a worse duplicate-program execution. Target: $C_5 = 0.380876$.

Capped feedback keeps improving



Cumulative best valid score; steps 0–2 omitted for late-stage resolution.

Feedback recovers

With the 16/4 cap, feedback reaches 0.381386 at step 5 and 0.381289 at step 10.

Late: memory

Memory-only lags early, then reaches the best observed tail, 0.381264, at step 11.

Equal budgets

The comparison fixes rollout count; it does not establish convergence.

The feedback cap works; two bottlenecks remain

What changed in feedback-only

- Teacher examples fell from 395 to 129 (-67%).
- Best C_5 improved from 0.381819 to 0.381289.
- Overall validity rose from 48.6% to 59.5%.

Memory selection still narrows

Memory-only tested 9/12 lessons but selected 3; both tested 13/14 but selected 2. Exploration works, but promotion is too sticky.

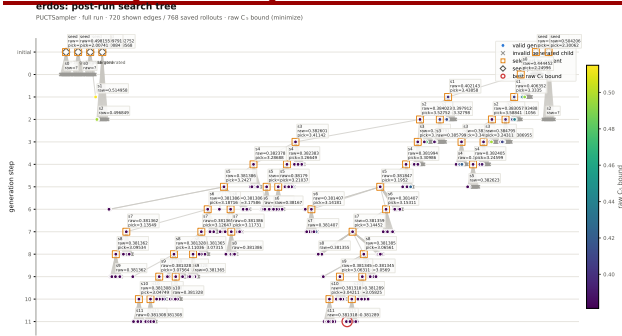
Interpretation

- With the same $\lambda_f = 0.2$, limiting repeated failures prevents repair from dominating reward.
- Matched $n = 3$ still makes memory credit noisy.
- Duplicate programs can keep an earlier, worse execution.

Next controlled test

Repeat capped feedback across seeds and rerun memory + feedback with the same 16/4 cap. Then discount stale memory evidence and retain the better duplicate execution.

Next step: adaptive rollout allocation



Fixed expansion creates wide 16-child fans; most children are evaluated once and never become parents.

What the tree reveals

Among 399 eligible pre-final children, only 39 (9.8%) were expanded later. Fixed 4×16 allocation gives only four informed choices per 64 rollouts.

Budgeted D-PUCT

Use microbatches of $b = 4$. After each one, choose $\textsc{Open}(p)$ (new sibling) or $\textsc{Descend}(p, c)$ (continue branch).

$$\widehat{\mathrm{EI}}_b(a) = \widehat{\mathbb{E}}\left[\left(\max_{j \le b} R_j(a) - r_t^\star\right)_+\right],$$

$$S_t(a) = \widehat{\mathrm{EI}}_b(a) + \beta_t U_t(a).$$

Progressive widening limits breadth: $|\mathcal{C}(p)| \le kN(p)^\alpha$, $0 < \alpha < 1$. Total compute stays at 64.